AWS VPC + RDS MySQL + Python CRUD Application

Final hands-on project report | October 2026

Project status: Completed based on successful tests reported during the lab. This document summarizes the implemented architecture and includes a reconstructed reference version of the Python application.

1. Project overview

Built a custom AWS network with a public EC2 jumpbox and a private Amazon RDS MySQL database. Installed Python and MySQL Connector/Python on the EC2 instance, then developed a command-line customer management application supporting Create, Read, Update and Delete (CRUD).

2. Architecture

```
MacBook / Terminal
       | SSH (TCP 22; restricted source)
       v
AWS VPC: mysql-lab-vpc (10.0.0.0/16)
  Public subnet: public-jumpbox-1a (10.0.1.0/24)
       | EC2: mysql-jumpbox / Python CRUD app
       | MySQL TCP 3306 (security-group controlled)
       v
  Private DB subnet: private-db-1a (10.0.11.0/24)
       | RDS MySQL: mysql-lab-db / labdb
  Additional private subnet: private-db-1b (10.0.12.0/24)
       | Used in DB subnet group across two AZs
```

The DB subnet group spans us-east-1a and us-east-1b; the RDS deployment itself is Single-AZ. RDS public access is disabled. The EC2 instance is the application host and administrative jumpbox.

3. AWS resource inventory

Resource	Configuration
Region	us-east-1 (N. Virginia)
VPC	mysql-lab-vpc | vpc-0b238388b2946a52c | 10.0.0.0/16
Public subnet	public-jumpbox-1a | 10.0.1.0/24 | us-east-1a
Private subnet A	private-db-1a | 10.0.11.0/24 | us-east-1a
Private subnet B	private-db-1b | 10.0.12.0/24 | us-east-1b
EC2	mysql-jumpbox | i-0862783157024ad43 | t3.micro | Amazon Linux 2023
EC2 security group	lab-jumpbox-sg
RDS	mysql-lab-db | MySQL 8.4.9 | db.t3.micro | Single-AZ
RDS storage	20 GiB gp2
RDS DB subnet group	mysql-lab-db-subnet-group

Resource	Configuration
RDS security group	lab-rds-sg
RDS database / port	labdb / TCP 3306
RDS public access	No

4. Database design and sample data

```
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    city VARCHAR(50),
    state VARCHAR(2),
    signup_date DATE
);
```

Initial test data consisted of John Smith (Fairfax, VA), Mary Jones (Arlington, VA), and Robert Brown (Alexandria, VA). A fourth test record, Susan Wilson, was inserted and subsequently updated; the delete operation was added and reported working.

5. Python environment and connection

```
ssh -i ~/Downloads/oracle-client-key.pem ec2-user@<CURRENT_EC2_PUBLIC_IP>
cd ~/mysql-python-app
source venv/bin/activate
python -m pip install mysql-connector-python
python -m py_compile crud_app.py
python crud_app.py
```

The SSH public IP may change when EC2 is stopped and restarted. The application prompts for the MySQL password with getpass rather than embedding it in source code. Do not store credentials in the repository.

6. Application capabilities and verification

Feature	Implementation	Reported outcome
Read	SELECT customers ORDER BY customer_id	Working
Create	INSERT with parameter placeholders; commit	Test customer added
Update	UPDATE city/state by customer_id; commit	Customer updated
Delete	DELETE by customer_id after yes/no confirmation	Reported working
Exit	Menu option 5 closes connection	Reported working

SQL statements use parameterized placeholders (%s) to separate user values from SQL text. The application commits successful writes, rolls back failed writes, and closes cursors and database connections.

7. Reference Python CRUD implementation

The following is a consolidated, reconstructed reference implementation based on the steps completed in this chat. It is not a byte-for-byte export of the EC2 file. The original application is stored on the EC2 instance as ~/mysql-python-app/crud_app.py.

```
import getpass
```

```
import mysql.connector

DB_HOST = "mysql-lab-db.c47ywe28idfx.us-east-1.rds.amazonaws.com"
DB_USER = "admin"
DB_NAME = "labdb"

def connect_database():
    password = getpass.getpass("Enter MySQL password: ")
    return mysql.connector.connect(
        host=DB_HOST, user=DB_USER, password=password,
        database=DB_NAME, port=3306, connection_timeout=10
    )

def display_customers(connection):
    cursor = connection.cursor()
    try:
        cursor.execute("SELECT * FROM customers ORDER BY customer_id")
        rows = cursor.fetchall()
        print("\nCUSTOMER LIST")
        for row in rows:
            print(row)
        print(f"Total customers: {len(rows)}")
    finally:
        cursor.close()

def add_customer(connection):
    first = input("First name: ").strip()
    last = input("Last name: ").strip()
    email = input("Email: ").strip()
    city = input("City: ").strip()
    state = input("State (2 letters): ").strip().upper()
    signup = input("Signup date (YYYY-MM-DD): ").strip()
    if not first or not last or len(state) != 2 or not state.isalpha():
        print("Invalid name or state.")
        return
    cursor = connection.cursor()
    try:
        cursor.execute(
            "INSERT INTO customers "
            "(first_name,last_name,email,city,state,signup_date) "
            "VALUES (%s,%s,%s,%s,%s,%s)",
            (first, last, email, city, state, signup)
        )
        connection.commit()
        print(f"Customer added successfully! ID: {cursor.lastrowid}")
    except mysql.connector.Error as error:
        connection.rollback()
        print(f"Insert failed: {error}")
    finally:
        cursor.close()

def update_customer(connection):
    try:
        customer_id = int(input("Customer ID: "))
    except ValueError:
        print("Invalid customer ID.")
        return
    cursor = connection.cursor()
    try:
        cursor.execute(
            "SELECT first_name,last_name,city,state "
            "FROM customers WHERE customer_id=%s", (customer_id,)
        )
        row = cursor.fetchone()
        if row is None:
            print("Customer not found.")
            return
        print(f"Customer: {row[0]} {row[1]} | {row[2]}, {row[3]}")
        city = input("New city: ").strip()
        state = input("New state (2 letters): ").strip().upper()
        if not city or len(state) != 2 or not state.isalpha():
            print("Invalid city or state.")
```

```
            return
        cursor.execute(
            "UPDATE customers SET city=%s,state=%s "
            "WHERE customer_id=%s", (city, state, customer_id)
        )
        connection.commit()
        print("Customer updated successfully!")
    except mysql.connector.Error as error:
        connection.rollback()
        print(f"Update failed: {error}")
    finally:
        cursor.close()

def delete_customer(connection):
    try:
        customer_id = int(input("Customer ID to delete: "))
    except ValueError:
        print("Invalid customer ID.")
        return
    cursor = connection.cursor()
    try:
        cursor.execute(
            "SELECT first_name,last_name FROM customers "
            "WHERE customer_id=%s", (customer_id,)
        )
        row = cursor.fetchone()
        if row is None:
            print("Customer not found.")
            return
        print(f"Customer: {row[0]} {row[1]}")
        if input("Delete this customer? (yes/no): ").strip().lower() != "yes":
            print("Deletion cancelled.")
            return
        cursor.execute(
            "DELETE FROM customers WHERE customer_id=%s", (customer_id,)
        )
        count = cursor.rowcount
        connection.commit()
        print("Customer deleted successfully!" if count == 1
              else "No customer was deleted.")
    except mysql.connector.Error as error:
        connection.rollback()
        print(f"Delete failed: {error}")
    finally:
        cursor.close()

def main():
    connection = None
    try:
        connection = connect_database()
        print("Connected to Amazon RDS MySQL!")
        while True:
            print("\n===== CUSTOMER MANAGEMENT =====")
            print("1. Display all customers")
            print("2. Add a new customer")
            print("3. Update a customer")
            print("4. Delete a customer")
            print("5. Exit")
            choice = input("Enter your choice: ").strip()
            if choice == "1":
                display_customers(connection)
            elif choice == "2":
                add_customer(connection)
            elif choice == "3":
                update_customer(connection)
            elif choice == "4":
                delete_customer(connection)
            elif choice == "5":
                print("Exiting application.")
                break
            else:
                print("Invalid choice.")
```

```
    except mysql.connector.Error as error:
        print(f"Database error: {error}")
    finally:
        if connection is not None and connection.is_connected():
            connection.close()
            print("Database connection closed.")


if __name__ == "__main__":
    main()
```

8. Operational and security notes

Keep RDS private and limit its security group inbound TCP 3306 to the EC2 security group. Limit EC2 SSH ingress to your trusted IP address. Avoid putting the RDS password in code or command history. For a production extension, use AWS Secrets Manager and a least-privilege database account rather than the RDS master account.

For a real deployment, also consider automated backups, monitoring, TLS verification for database connections, better input validation, and an application user with only the required SQL permissions.

9. Troubleshooting and lessons learned

The DB subnet group initially required correction to include subnets in two Availability Zones. The Linux environment provided the mariadb client for manual testing. Python pip was installed with dnf; a virtual environment isolated the connector dependency. A Python indentation error was corrected before successful execution. Menu numbering was updated as features were added.

10. Portfolio-ready project summary

Designed and deployed a custom AWS VPC with public and private subnets, configured an EC2 Linux jumpbox and private Amazon RDS MySQL instance, and built a Python command-line CRUD application for customer data. Implemented parameterized SQL queries, transaction handling, password prompting, and security-group-based database access. Validated customer retrieval, insertion, updates, and deletion.

Technologies: AWS VPC, EC2, RDS MySQL, Amazon Linux 2023, Python 3, MySQL Connector/Python, SQL, SSH, security groups, and Linux CLI.

Evidence note: Completion and test results are based on the user's reports during this guided lab, not independently inspected AWS console logs or a direct copy of the EC2 filesystem.